Lo Scheduling

Cos'è lo Scheduling?

Lo scheduling nei sistemi operativi è il meccanismo mediante il quale il sistema decide quale processo, tra quelli pronti all'esecuzione, debba ottenere l'uso della CPU e per quanto tempo. Nei sistemi multiprogrammati e time-sharing, in cui più processi competono simultaneamente per l'utilizzo del processore, lo scheduling rappresenta un meccanismo fondamentale perché consente di:

- evitare che un singolo processo monopolizzi la CPU;
- garantire la condivisione delle risorse;
- fornire l'illusione di esecuzione concorrente;
- assicurare tempi di risposta adeguati nei sistemi interattivi.

Ruolo e obiettivi dello scheduler

Lo scheduler è il componente del sistema operativo (parte del kernel) responsabile della selezione del prossimo processo da eseguire tra quelli nello stato di Ready.

I principali obiettivi dello scheduling sono:

- *Fairness (equità):* garantire una distribuzione equa del tempo di CPU tra i processi, evitando starvation.
- *Throughput:* massimizzare il numero di processi completati nell'unità di tempo.
- *Tempo di risposta (reattività):* minimizzare il tempo che intercorre tra una richiesta dell'utente e la prima risposta del sistema.
- *Tempo di attesa:* ridurre il tempo trascorso dai processi nello stato di Ready.

Diagramma degli stati di un processo

Un processo, durante il suo ciclo di vita, può trovarsi in diversi stati. Nel modello semplificato a tre stati si distinguono:

- *Ready:* il processo è pronto per essere eseguito e attende l'assegnazione della CPU.
- *Running:* il processo è attualmente in esecuzione sulla CPU.
- *Waiting (o Blocked):* il processo è in attesa del completamento di un evento esterno (ad esempio un'operazione di I/O).

Le principali transizioni tra stati sono:

- Ready → Running: lo scheduler assegna la CPU al processo.
- Running → Waiting: il processo richiede un'operazione di I/O o attende un evento.
- Waiting → Ready: l'evento atteso si verifica (es. completamento I/O).
- Running → Ready: il processo viene interrotto (preemption) per consentire l'esecuzione di un altro processo.

In modelli più completi si considerano anche gli stati New e Terminated.

Scheduling preemptive e non preemptive

Si distinguono due principali modalità di scheduling:

- **Non preemptive:** una volta assegnata la CPU, il processo la mantiene fino al completamento o fino a quando entra volontariamente nello stato di Waiting. Il sistema operativo non interrompe forzatamente il processo.
- **Preemptive:** il sistema operativo può interrompere un processo in esecuzione (ad esempio tramite un interrupt generato da un timer) e assegnare la CPU a un altro processo. Il processo interrotto torna nello stato di Ready.

Lo scheduling preemptive è tipico dei sistemi time-sharing, poiché consente maggiore equità e migliore reattività.

Algoritmi per Sistemi Inlterattivi e Batch

Per **SISTEMI BATCH** intendiamo sistemi che elaborano grandi volumi di dati senza intervento dell'utente ("umano"). Conosciamo 3 algoritmi per i sistemi Batch:

- **First-Come, First-Served**: È un algoritmo *Non Preemptive*. I processi vengono assegnati alla CPU nell'ordine in cui arrivano. Per gestire ciò viene usata una singola coda. Quando il processo in esecuzione si blocca viene eseguito il successivo.
  - VANTAGGI: Facile da capire e da implementare
  - SVANTAGGI: Tempi di attesa molto lunghi
- **Shortest Job First:** È un algoritmo *Non Preemptive*. Supponendo che i tempi di eseuzione dei job siano noti in anticipo. Verrà scelto il job più breve.
  - VATAGGI: minimizza il *turnaround time* (tempo di completamento) *medio*
  - SVANTAGGI: L'algoritmo non è ottimale se i job arrivano in momenti diversi. Essendo un algoritmo *Non Preemptive* , se un processo lungo impegna la CPU per primo, lo scheduler non può interromperlo anche se arriva un secondo processo con un tempo di esecuzione inferiore.
- **Shortest Remaining Time Text:** È un algoritmo *Preemptive*. Suppenendo che il tempo dei job sia noto in anticipo. Lo Scheduler sceglie sempre il processo che impiegherà meno tempo per l'esecuzione. Ma lo fa confrontando il tempo del nuovo job con il tempo restante dei processi attuali. Se il nuovo job richiede meno tempo, lo scheduler blocca il job attuale e esegue il nuovo.

Per **SISTEMI INTERATTIVI** intendiamo sistemi che permettono lo scambio diretto e in tempo reale tra utente e computer. Conosciamo 6 algoritmi per i sistemi Interattivi:

- **Round Robin Scheduling:** Lo scheduler con una lista (spesso una *linked list*) di processi pronti a partire. Ad ogni processo viene assegnato un "quanto" (quantità/intervallo di tempo). Prende il primo processo in testa alla lista. Lo esegue per un "quanto" di tempo. Se il processo finisce i suoi calcoli prima che scada il "quanto", libera la CPU spontaneamente. Altrimenti, se non ha finito, lo interrompe e lo mette in fondo alla lista d'attesa (in fondo alla coda) e passa al processo successivo.

- **Priority Scheduling:** Ad ogni processo viene assegnata una priorità (un numero). Lo scheduler guarda la lista di processi pronti e esegue quello con priotità più alta. Se un processo con priorità più alta diventa pronto mentre uno con priorità più bassa sta girando, lo scheduler interrompe il processo che stava eseguendo con priorità più basso per far partire quello con priorità più alta.

- **Shortest Process Next:** È simile all'algoritmo Shortest Job First per i sistemi Batch, con una particolarità in più. Noi non sappiamo il tempo di stima per ogni processo. Quindi in base ai tempi di esecuzione di ogni processo calcoliamo una stima, in modo tale che lo scheduler sceglierà il processo con stima minore. La stima verrà calcolata nel seguente modo:
$$S_{i+1} = aS_i + (1 - a)T_i$$
dove:
  - S_i stima corrente
  - a numero qualsiasi da 0 a 1, che decido per calcolare la stima

- **Guaranteed Scheduling:** Se ci sono n processi, ognuno deve ricevere esattamente *1/n* della potenza totale della CPU. Il sistema tiene traccia di due valori fondamentali:
  - *Tempo Consumato:* Il tempo effettivo che il processo ha passato dentro la CPU dal momento in cui è nato.
  - *Tempo Dovuto:* Quanto tempo il processo avrebbe dovuto ricevere idealmente. Si calcola dividendo il tempo trascorso dalla sua creazione per il numero totale di processi n

  Il sistema si calcola il rapporto:
$$\text{Rapporto} = \frac{\text{Tempo Dovuto}}{\text{Tempo Consumato}}$$
  Se questo rapporto è ≤ 1.0 allora il processo dovrebbe ricevere più CPU.
  Se questo rapporto è ≥ 1.0 allora il processo ha sfruttato più CPU di quella che gli serviva.

- **Lottery Scheduling:** Il sistema operativo emetta un numero totale di n biglietti numerati.
  Se un processo è molto importante, riceve una quantità di biglietti maggiore.
  Se un processo è meno importante, ne riceve meno. Ogni volta che la CPU deve decidere che processo eseguire, "estrae" un numero a sorte. Chi possiede quel numero vince il turno di esecuzione.
  Questo algoritmo ha 3 proprietà fondamentali:
  - Reattività: un processo con una quantità maggiore di biglietti ha più probabilità di essere eseguito, senza fare lunghe "attese".

  - I processi possono scambiarsi processi tra loro
  - è flessibile, possiamo "regolare" la priorità dei processi.

- **Fair-Share Scheduling:** Supponiamo di avere 2 utenti, ognuno possiede un numero di processi. Assegniamo la stessa quantità di CPU ad entrambi gli utenti. Quindi ogni utente divide la stessa quantità di CPU tra il numero di processi (che può essere diverso). **NB:** Se entrmabi gli utenti hanno lo stesso numero di processi, verranno eseguiti nello stesso tempo e con la stessa quantità di CPU. Allora i processi sono equi-probabili.

Algoritmi per Sistemi Real Time e Thread

I **SISTEMI REAL TIME**, sono sistemi operativi sviluppati per il supporto di applicazioni software real time.
Questi sistemi possono essere *Preemptive* o non *Non Preemptive*. I processi sanno che non possono essere eseguiti per lunghi periodi di tempo, infatti generalmente vengono eseguiti e si bloccano rapidamente. Hanno i seguenti obiettivi:

- Rispettare le scadenze
- Assicurarsi che il funzionamento sia costante

Questi algoritmi si usano in applicazioni in cui il tempo di riaposta è fondamentale (es: lettori musicali, monitoraggio in terapia intensiva, ecc..).
Possiamo dividerli in 2 categorie:

- **Hard Real-Time:** quando si tratta di scadenze assolute da rispettare
- **Soft Real-Time:** quando se una scadenza non viene rispettata, di tanto in tanto, c'è un certo grado di tollerabilità

Dati m processi periodici, ciascuno caratterizzato da un periodo di attivazione P_i (intervallo regolare di ripetizione) e da un tempo di elaborazione C_i (risorse CPU richieste per singola istanza), il carico di lavoro è calcolato nel seguente modo:

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$$

.
Questi algoritmi posso pure essere:

- **Statici:** questi prendono decisioni di scheduling prima che il sistema inizi l'esecuzione
- **Dinamici:** questi prendono decisione di scheduling durante l'esecuzione

Nei **SISTEMI THREAD**, un singolo processo può essere suddiviso in più unità di esecuzione indipendenti (i thread) che lavorano in parallelo condividendo però le stesse risorse e la stessa memoria del processo padre.
Esistono due tipi di Scheduling di Thread:

- **Thread a Livello Utente:** qui il kernel ignora l'esistenza dei thread, infatti sceglie un processo per il suo quanto. Lo scheduler interno al processo decide quale thread eseguire. Questo porta ad avere thread che possono consumare l'intero quanto del processo, influenzando solo il processo interno e non gli altri. Lo scambio dei thread avviene con poche istruzioni, poichè ci pensa il processo al suo interno.

- **Thread a Livello Kernel:** qui il kernel seleziona un thread specifico per l'esecuzione; se il tempo di esecuzione di un thread supera il quanto, allora viene sospeso. Lo scambio del thread comporta uno scambio di contesto, quindi è più lento.